

http://algs4.cs.princeton.edu

3.2 BINARY SEARCH TREES

- *BSTs*
- *ordered operations*
- *deletion*

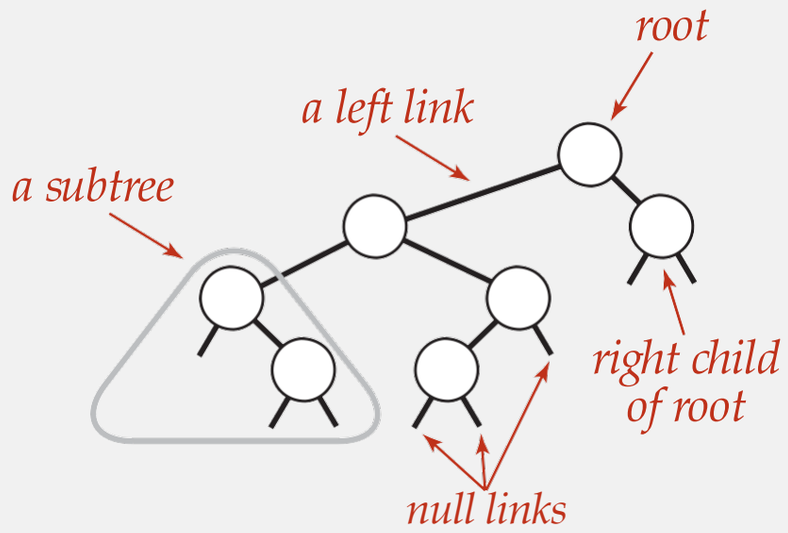


3.2 Binary Search Trees

- *BSTs*
- *ordered operations*
- *deletion*

Binary search trees

Definimi. Një BST është binary tree me rradhitje simetrike.

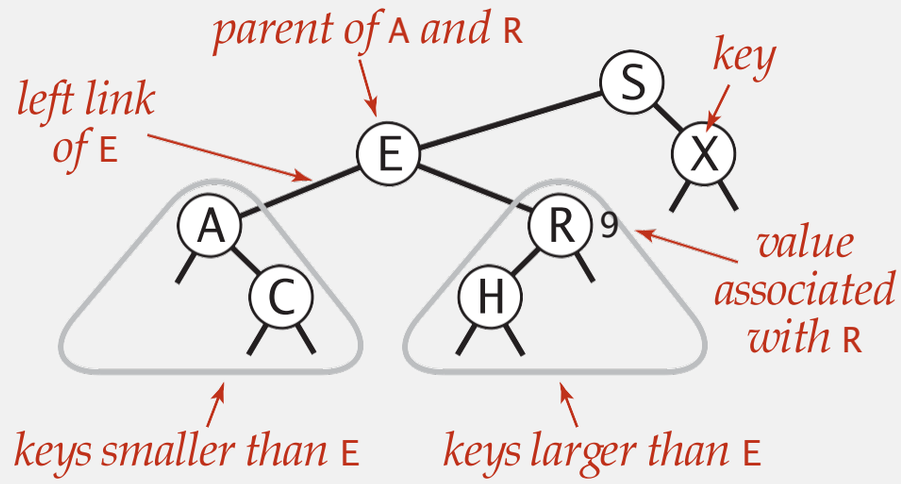


Një binary tree mund të jetë ose:

- i zbrazët.
- dy binary trees të veçantë (majtë dhe djathtë).

Rradhitja Simetrike. Çdo node ka një çelës, dhe çdo çelës i nyjës është:

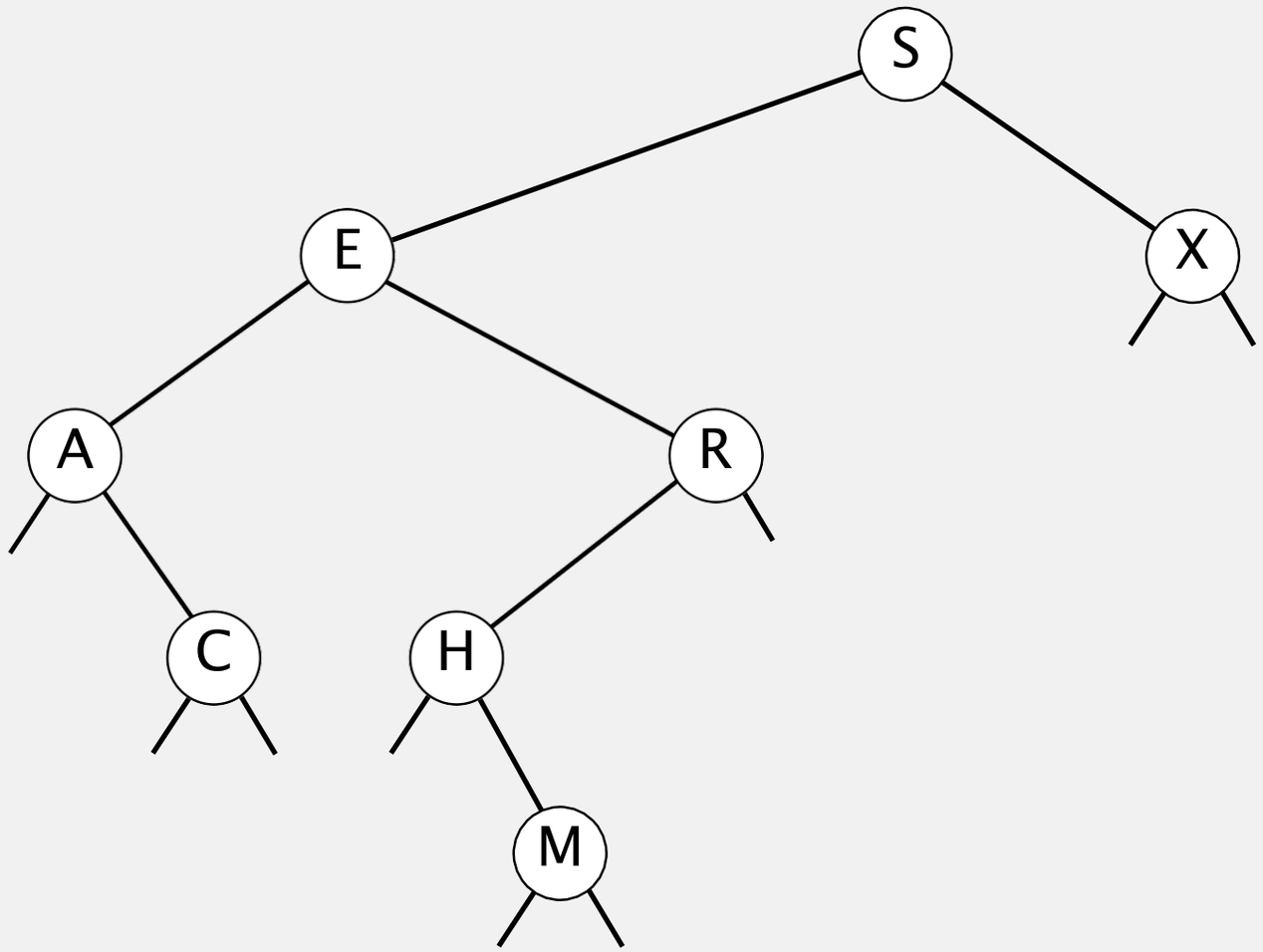
- më i madh sesa të gjithë çelësat në subtree të majtë.
- më i vogël sesa të gjithë çelësat në subtree të djathtë.



Binary search tree demo

Search. Nëse më i vogël (less), shkohet majtas; nëse më i madh (greater), shkohet djathtas; nëse baraz (equal), search hit.

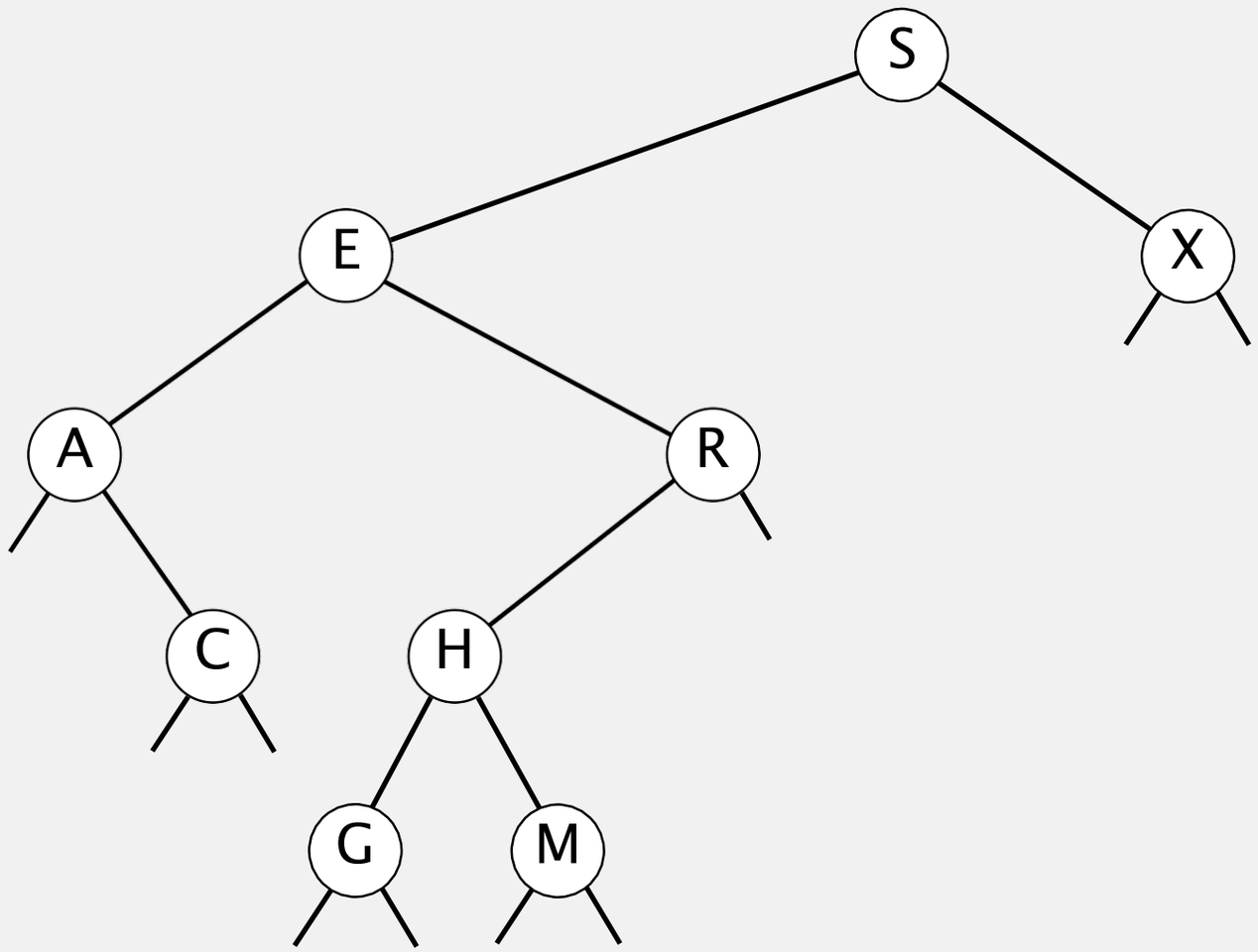
search i suksesshëm për H



Binary search tree demo

Insert. Nëse less, shkohet majtas; nëse greater, shkohet djathtas; nëse null, insert.

insert G



BST paraqitja në Java

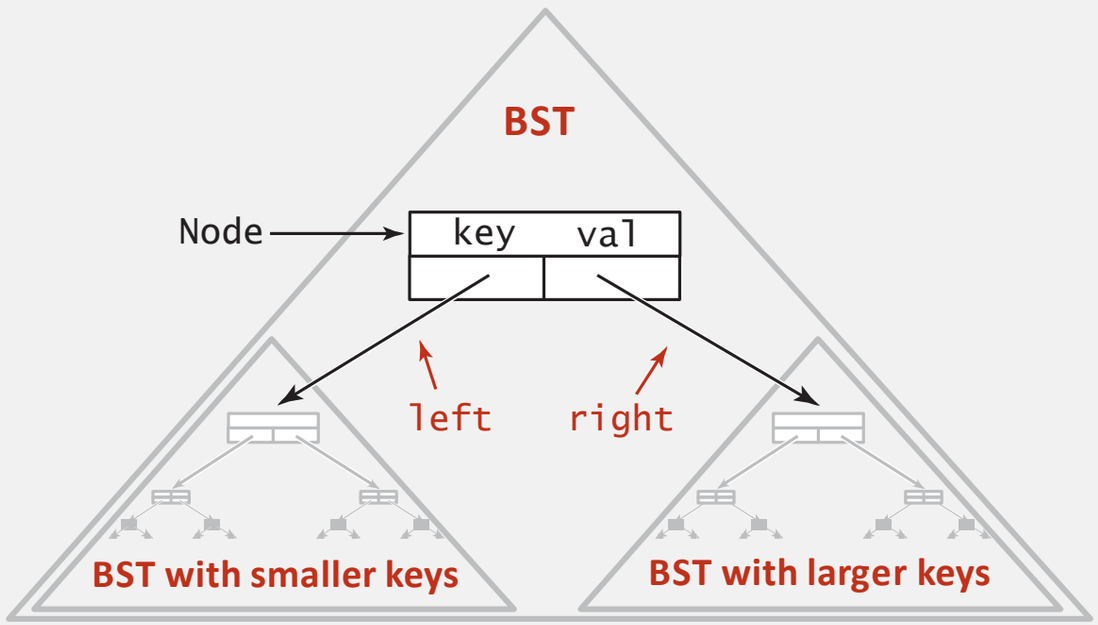
Java definimi. Një BST është një referencë e një root `Node`.

Një `Node` është përbërë prej 4 fushave:

- Një `Key` dhe një `Value`.
- Një referencë për subtree të majtë dhe

një reference për subtree të djathtë.

```
private class Node
{
   private Key key;
   private Value val;
   private Node left, right;
   public Node(Key key, Value val)
   {
      this.key = key;
      this.val = val;
   }
}
```



Binary search tree

Key dhe Value janë tipe gjenerike; Key është Comparable

BST implementimi (skeleti)

```
public class BST<Key extends Comparable<Key>, Value>
{
    private Node root;

   private class Node
   {  /* see previous slide */  }

   public void put(Key key, Value val)
   {  /* see next slides */  }

   public Value get(Key key)
   {  /* see next slides */  }

   public void delete(Key key)
   {  /* see next slides */  }

   public Iterable<Key> iterator()
   {  /* see next slides */  }

}
```

root of BST

BST search: Java implementimi

Get. Return vlerën që korrespondon me çelësin përkatës, ose `null` nëse nuk ka asi lloj çelësi.

```
public Value get(Key key)
{
   Node x = root;
   while (x != null)
   {
      int cmp = key.compareTo(x.key);
      if      (cmp  < 0) x = x.left;
      else if (cmp  > 0) x = x.right;
      else if (cmp == 0) return x.val;
   }
   return null;
}
```

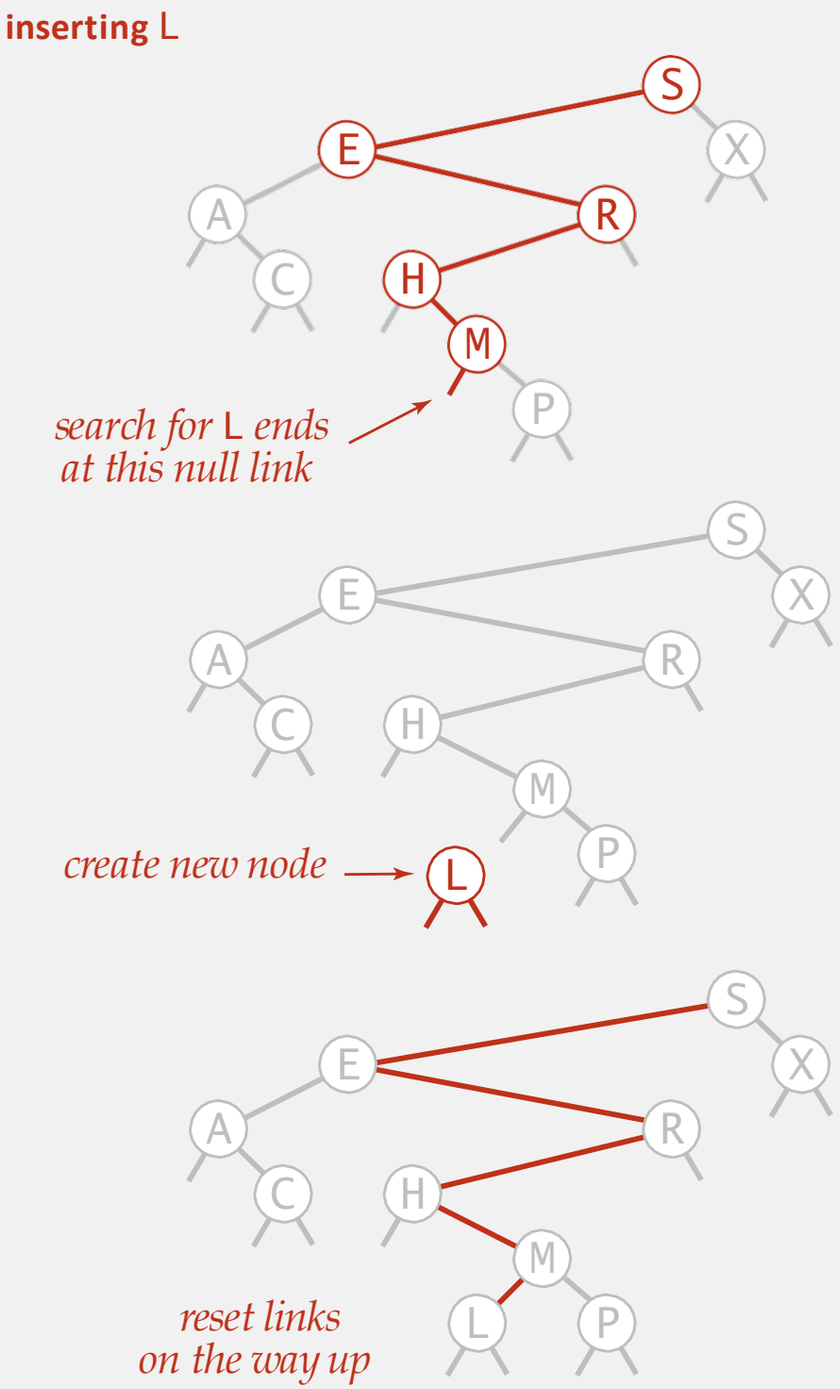
Çmimi. Numri i krahasimeve është baraz me 1 + thellësia e nyjës në dru.

BST insert

Put. Asociohet vlera me çelës.

Search për çelës, ka dy raste të mundshme:

- Çelësi në tree reset vlerën.
- Çelësi nuk është në tree ⇒ add nyje të re - new node.



Insertion into a BST

BST insert: Java implementimi

Put. Associohet vlera me çelës.

concise, but tricky, recursive code; read carefully!

```
public void put(Key key, Value val)
{  root = put(root, key, val);  }

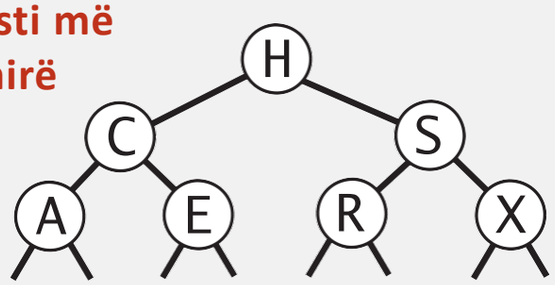
private Node put(Node x, Key key, Value val)
{
   if (x == null) return new Node(key, val);
   int cmp = key.compareTo(x.key);
   if      (cmp  < 0)
      x.left  = put(x.left,  key, val);
   else if (cmp  > 0)
      x.right = put(x.right, key, val);
   else if (cmp == 0)
      x.val = val;
   return x;
}
```

Çmimi. Numri i krahasimeve është baraz me 1 + thellësia e nyjës në dru.

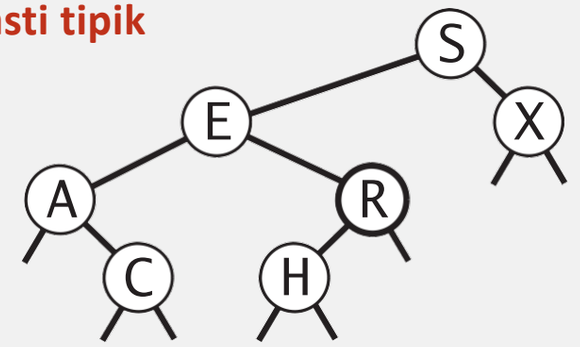
Forma e drurit (tree)

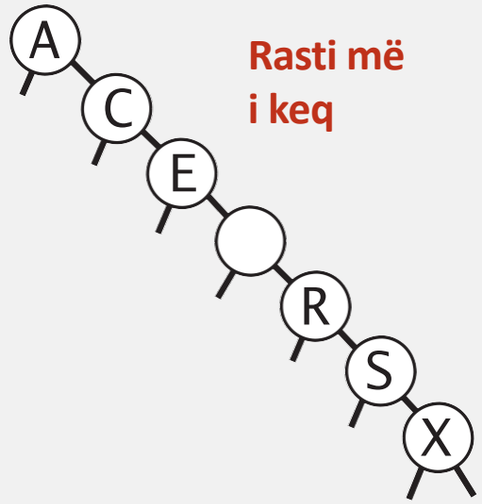
- Shumë BSTs korrespondojnë me të njejtën bashkësi të çelësave
- Numri i krahasimeve për search/insert është baraz me 1 + thellësia e nyjës.

Rasti më i mirë



Rasti tipik



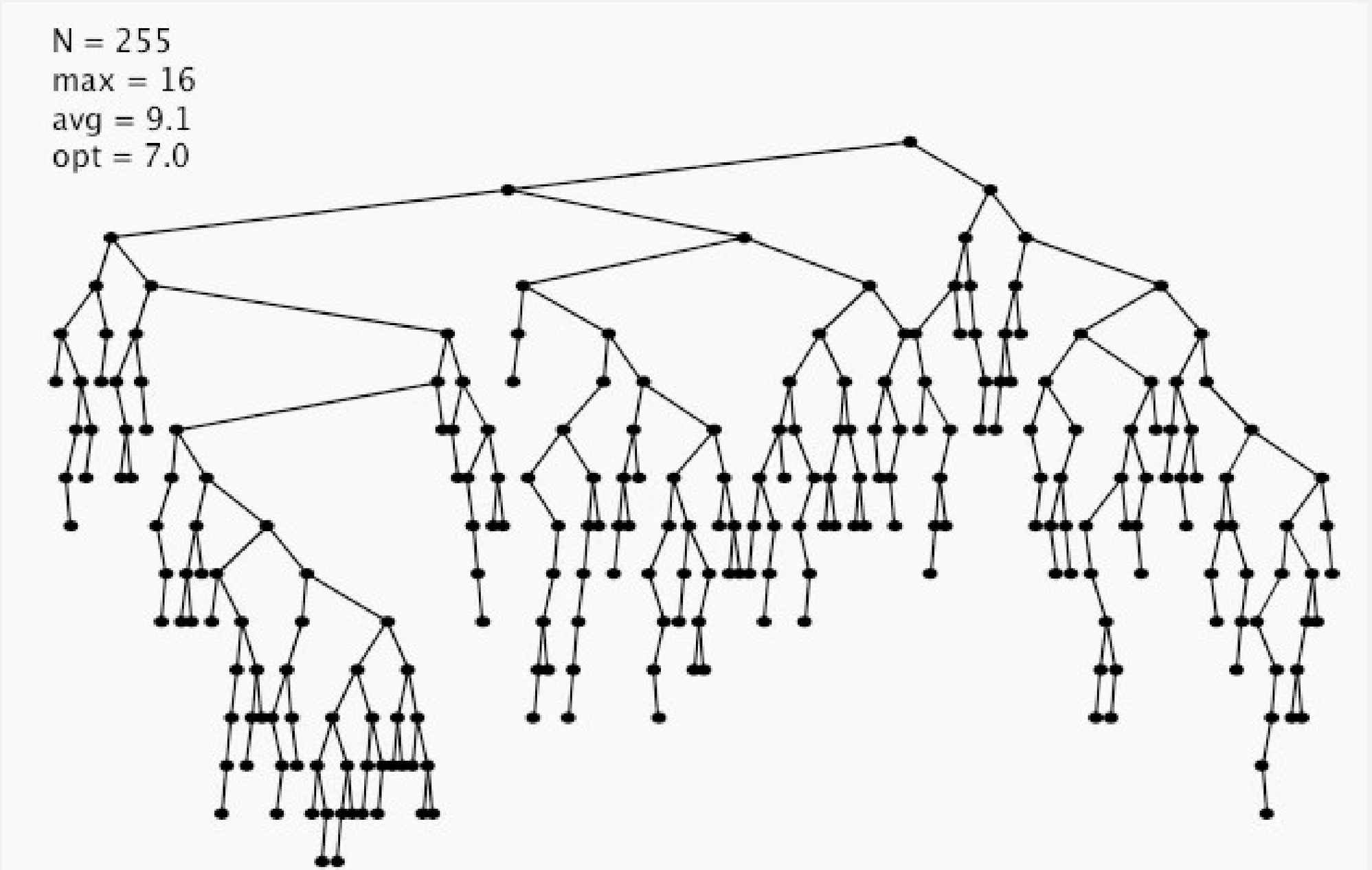


Rasti më i keq

Përfundim. Forma e drurit varet nga rradha e insertimeve.

BST insertimi: Shembull i përgjithshëm

P.sh. Insert çelësa me rradhitje të rastësishme.



ST implementimi: përmbledhje

implementation	guarantee		average case		operations on keys
	search	insert	search hit	insert	
sequential search (unordered list)	N	N	$\frac{1}{2}\ N$	N	`equals()`
binary search (ordered array)	$\lg N$	N	$\lg N$	$\frac{1}{2}\ N$	`compareTo()`
BST	N	N	$1.39 \lg N$	$1.39 \lg N$	`compareTo()`



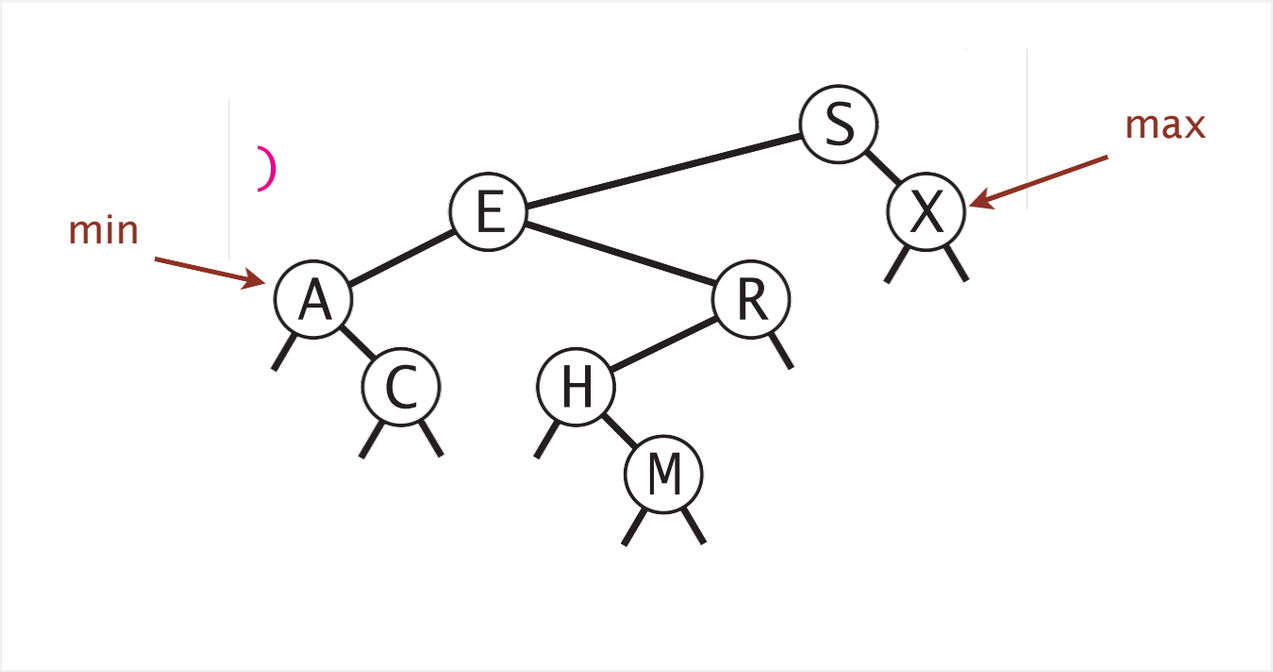
3.2 Binary Search Trees

- BSTs
- *ordered operations*
- deletion

Minimumi dhe maksimumi

Minimum. Çelësi më i vogël në tabelë.

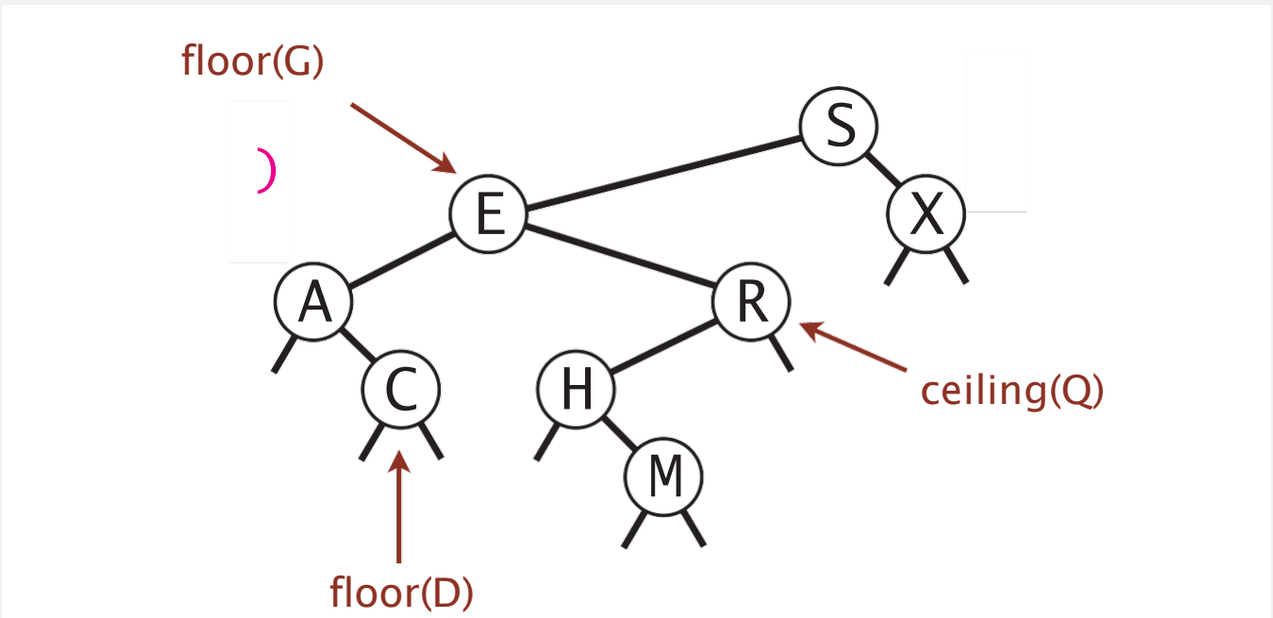
Maximum. Çelësi më i madh në tabelë.



Kufijtë e poshtëm dhe i lartë

Floor. Çelësi më i madh $\leq$ një çelës.

Ceiling. Çelësi më i vogël $\geq$ një çelës.



Llogaritja e floor (kufiri i poshtëm)

Rasti 1. [k equals the key in the node]

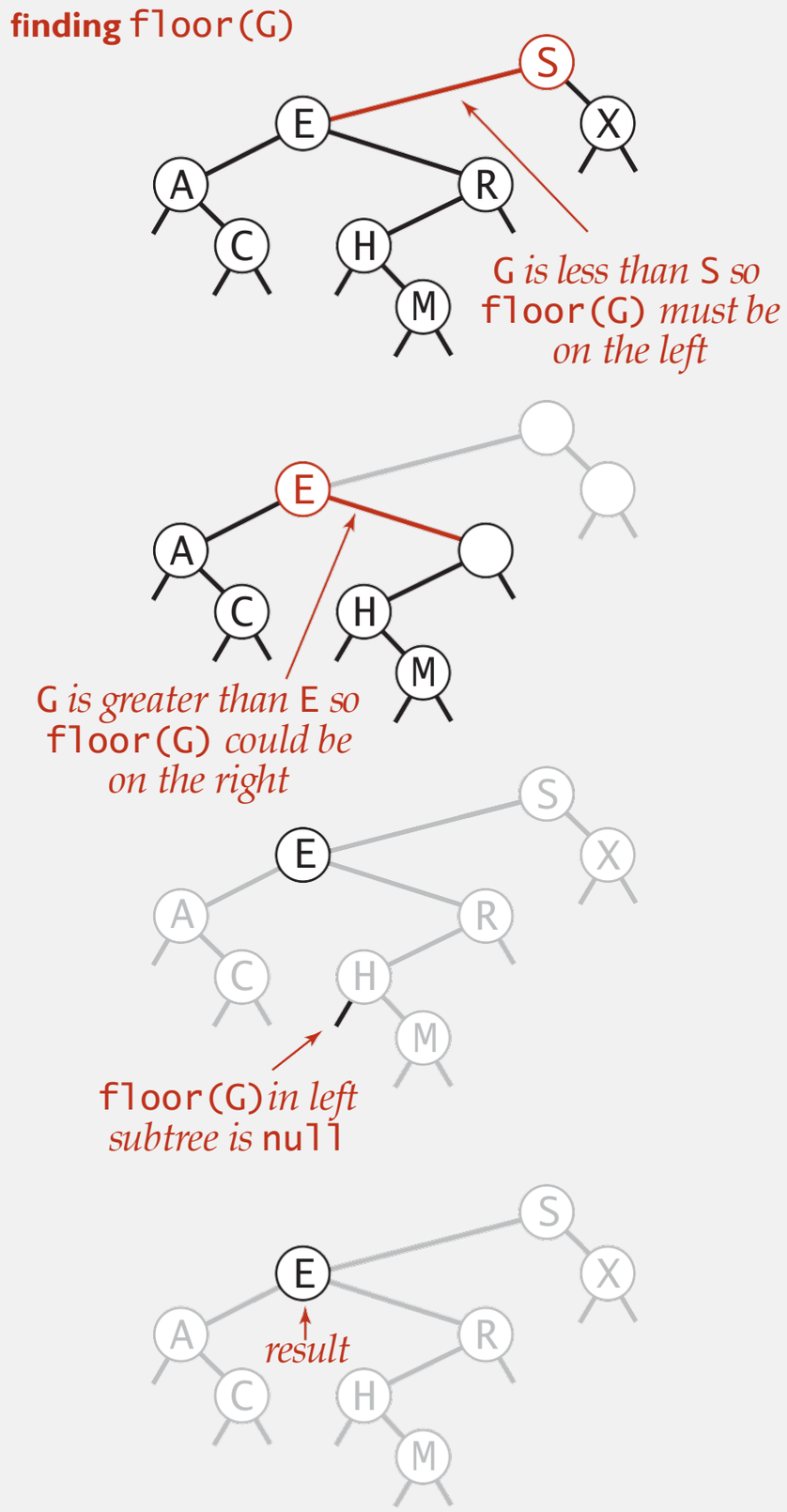
The floor i k është k.

Rasti 2. [k is less than the key in the node]

The floor of k është në të majtë të subtree.

Rasti 3. [k is greater than the key in the node]

The floor i k është në të djathtë të subtree (nëse ka ndonjë key $\leq k$ në subtree të djathtë); përndryshe është çelësi në nyjën në fjalë.



Llogaritja e floor (kufiri i poshtëm)

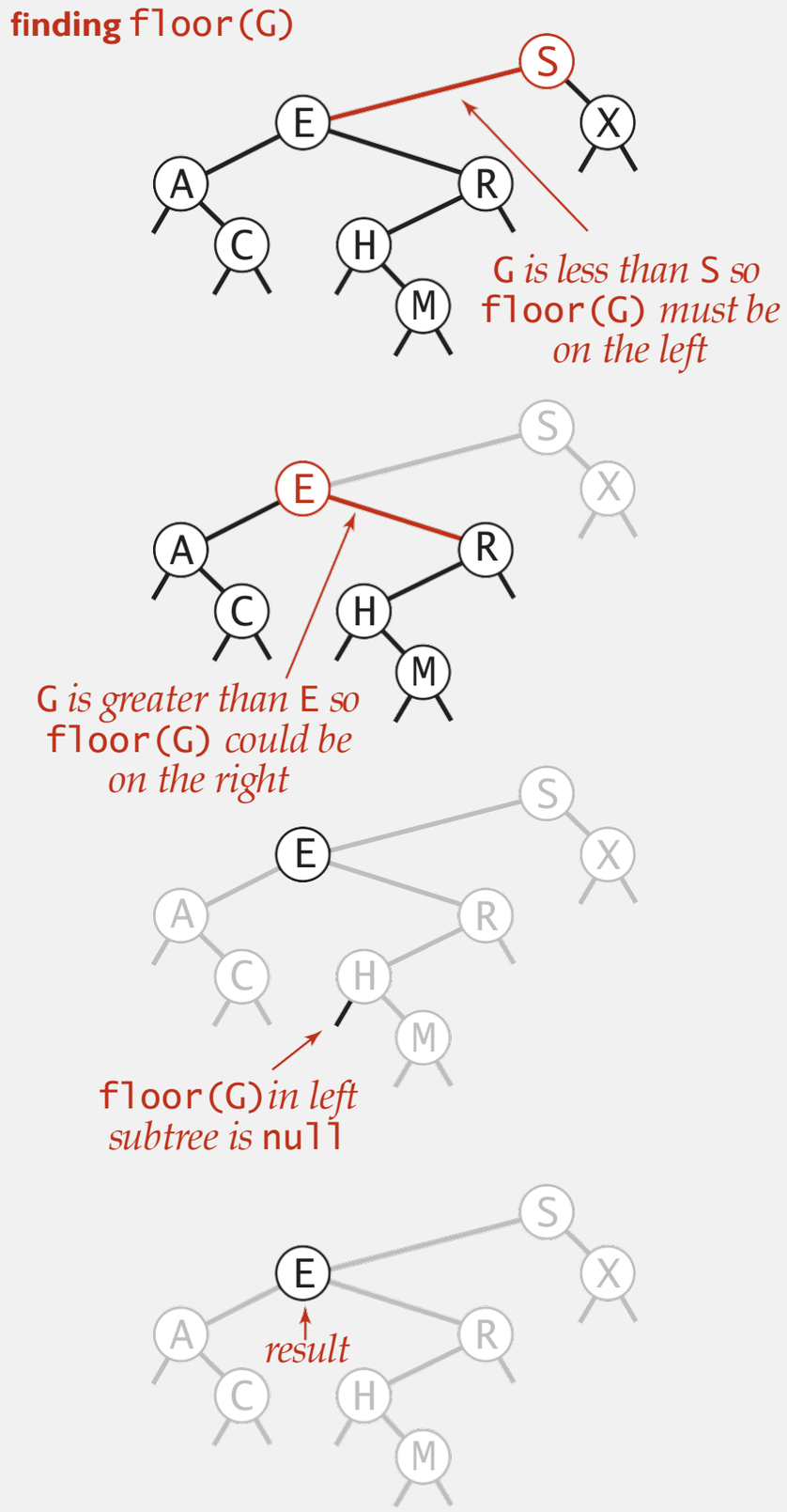
```
public Key floor(Key key)
{
   Node x = floor(root, key);
   if (x == null) return null;
   return x.key;
}
private Node floor(Node x, Key key)
{
   if (x == null) return null;
   int cmp = key.compareTo(x.key);

   if (cmp == 0) return x;

   if (cmp < 0)  return floor(x.left, key);

   Node t = floor(x.right, key);
   if (t != null) return t;
   else           return x;

}
```



BST implementimi: subtree count-at

```
private class Node
{
   private Key key;
   private Value val;
   private Node left;
   private Node right;
   private int count;
}
```

number of nodes in subtree

```
public int size()
{  return size(root);  }

private int size(Node x)
{
   if (x == null) return 0;
   return x.count;
}
```

ok to call when x is null

```
private Node put(Node x, Key key, Value val)
{
   if (x == null) return new Node(key, val, 1);
   int cmp = key.compareTo(x.key);
   if      (cmp  < 0) x.left  = put(x.left,  key, val);
   else if (cmp  > 0) x.right = put(x.right, key, val);
   else if (cmp == 0) x.val = val;
   x.count = 1 + size(x.left) + size(x.right);
   return x;
}
```

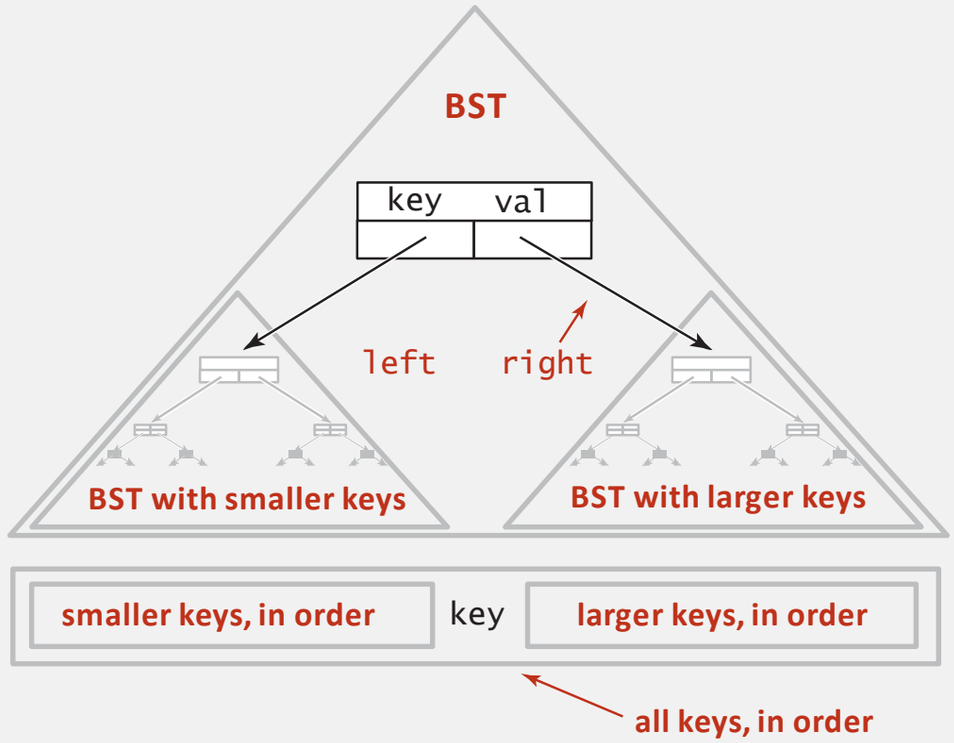
initialize subtree count to 1

Inorder traversal

- Traverse në subtree të majtë.
- Enqueue key.
- Traverse në subtree të djathtë.

```
public Iterable<Key> keys()
{
    Queue<Key> q = new Queue<Key>();
    inorder(root, q);
    return q;
}

private void inorder(Node x, Queue<Key> q)
{
   if (x == null) return;
   inorder(x.left, q);
   q.enqueue(x.key);
   inorder(x.right, q);
}
```



Veçori. Inorder traversal i një BST jep çelësat e rradhituar nga më i madhi deri tek më i vogli

BST: Tabela e operacioneve për symbol table të sortuar - përmbledhje

	sequential search	binary search	BST
search	N	$\lg N$	h
insert	N	N	h
min / max	N	1	h
floor / ceiling	N	$\lg N$	h
rank	N	$\lg N$	h
select	N	1	h
ordered iteration	$N \log N$	N	N

h = height of BST
(proportional to log N
if keys inserted in random order)

order of growth of running time of ordered symbol table operations



3.2 Binary Search Trees

- BSTs
- ordered operations
- deletion

ST implementations: Përmbledhje

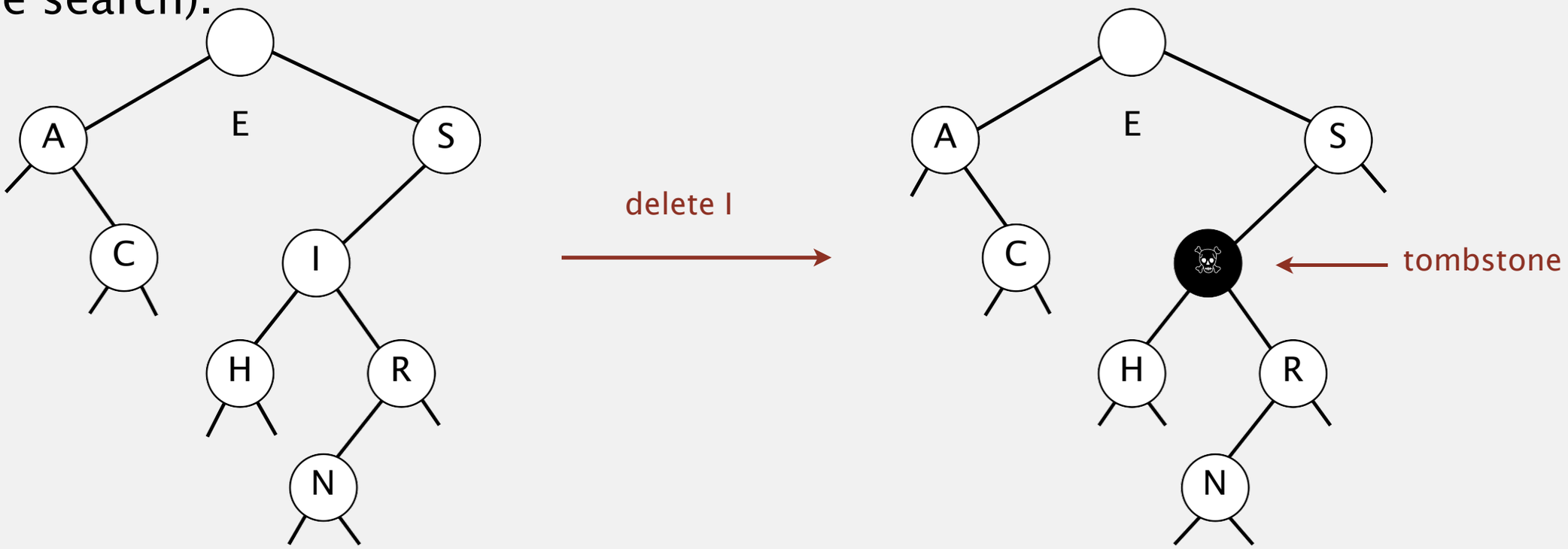
implementation	guarantee			average case			ordered ops?	operations on keys
	search	insert	delete	search hit	insert	delete		
sequential search (linked list)	N	N	N	$\frac{1}{2}\ N$	N	$\frac{1}{2}\ N$		`equals()`
binary search (ordered array)	$\lg N$	N	N	$\lg N$	$\frac{1}{2}\ N$	$\frac{1}{2}\ N$	✓	`compareTo()`
`BST`	N	N	N	$1.39 \lg N$	$1.39 \lg N$	? ? ?	✓	`compareTo()`

Next. Delete tek BSTat.

BST delete: Qasja e përtacit (lazy approach)

Për të remove një nyje me një çelës specific:

- Set vlerën e tij në `null`.
- Lihet çelësi në dru për të ndihmuar search (por nuk konsiderohet pjesë e search).



Çmimi. $\sim 2 \ln N'$ për insert, search, dhe delete (kur çelësat janë random).

Ku N' është numri i çiftit çelës-vlerë që janë insertuar në BST.

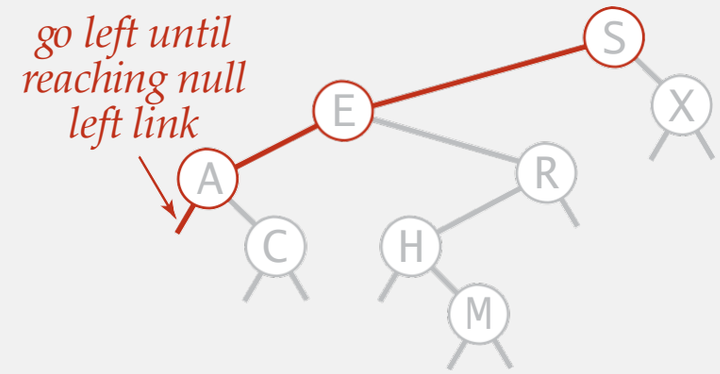
Zgjidhje jo aq e kënaqshme.

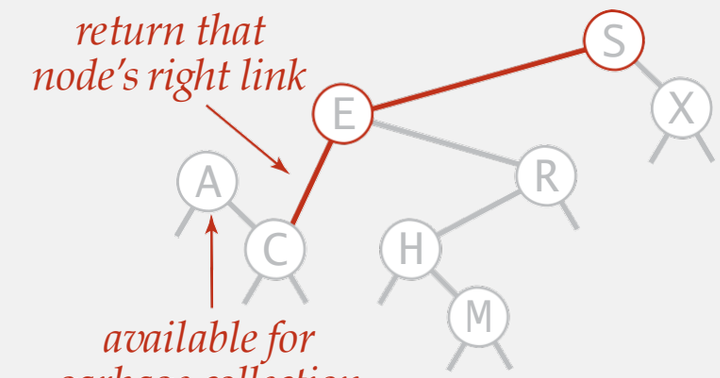
Tombstone (memory) overload.

Delete minimum-in

Për të delete çelësin minimal

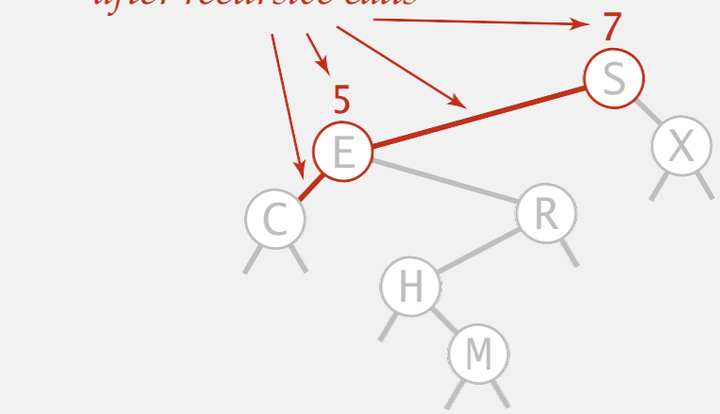
- Shkohet majtas derisa të gjindet një nyje me fëmijë të majtë null.
- Zëvendësohet ajo nyje me fëmijën e djathtë.
- Update subtree counts.






```
public void deleteMin()
{  root = deleteMin(root);  }

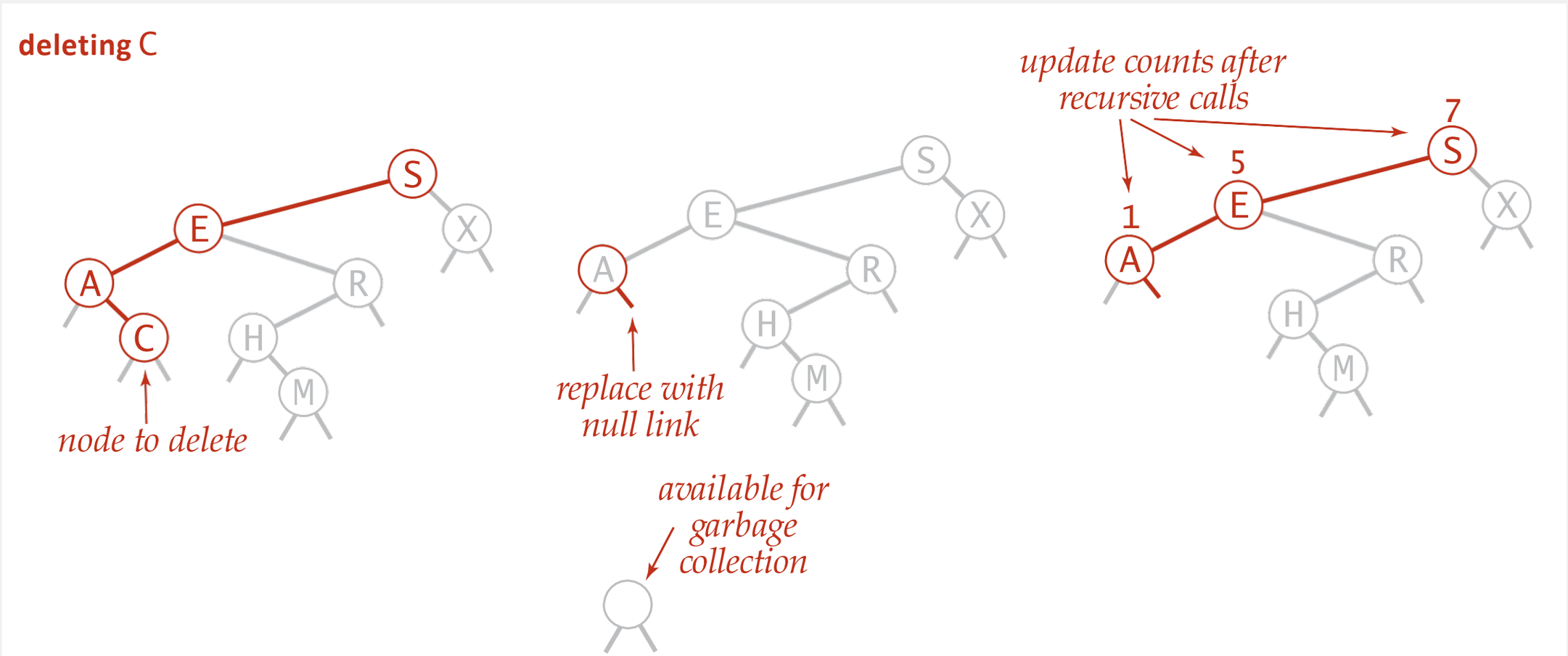
private Node deleteMin(Node x)
{
   if (x.left == null) return x.right;
   x.left = deleteMin(x.left);
   x.count = 1 + size(x.left) + size(x.right);
   return x;
}
```



Hibbard delete

Për të delete një nyje me çelës k: search për nyjën t që përmbanë çelësin k.

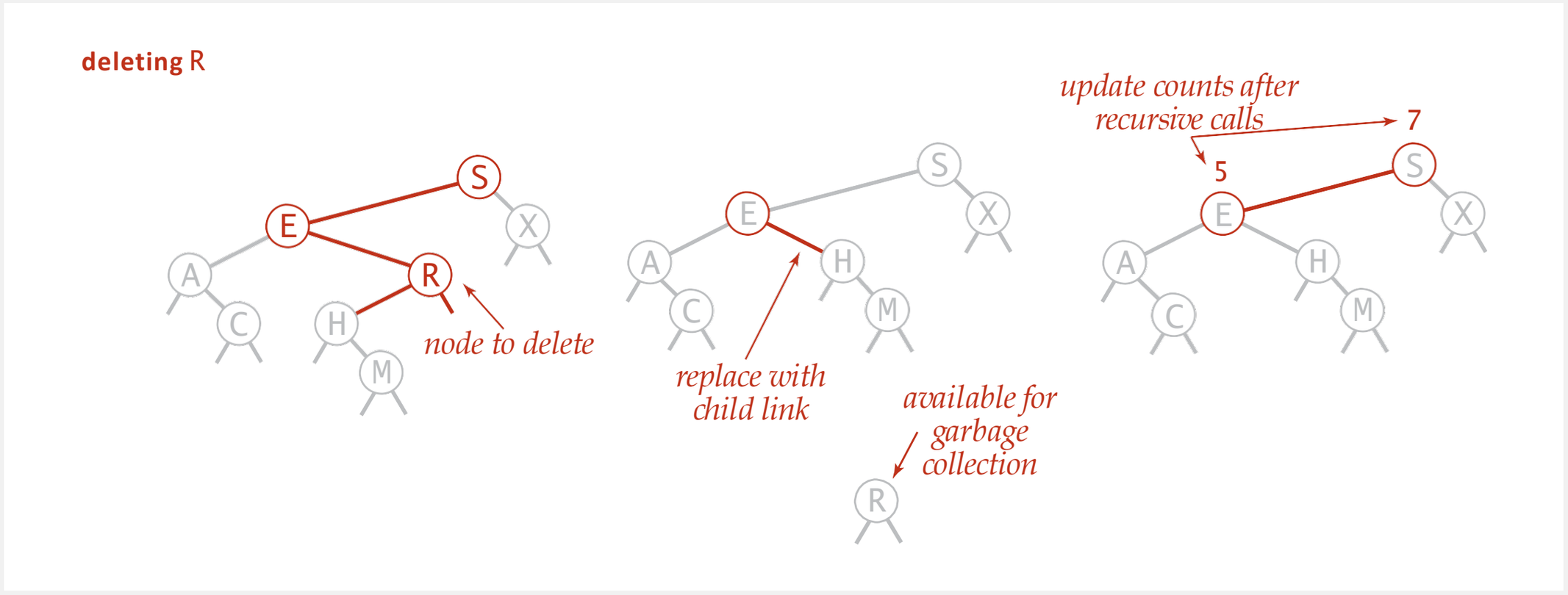
Rasti 0. [0 fëmijë] Delete t duke set nyjën prind në null.



Hibbard delete

Për të delete një nyje me çelës `k`: search për nyjën `t` që përmbanë çelësin `k`.

Rasti 1. [1 fëmijë] Delete `t` duke zëvendësuar nyjën prind.

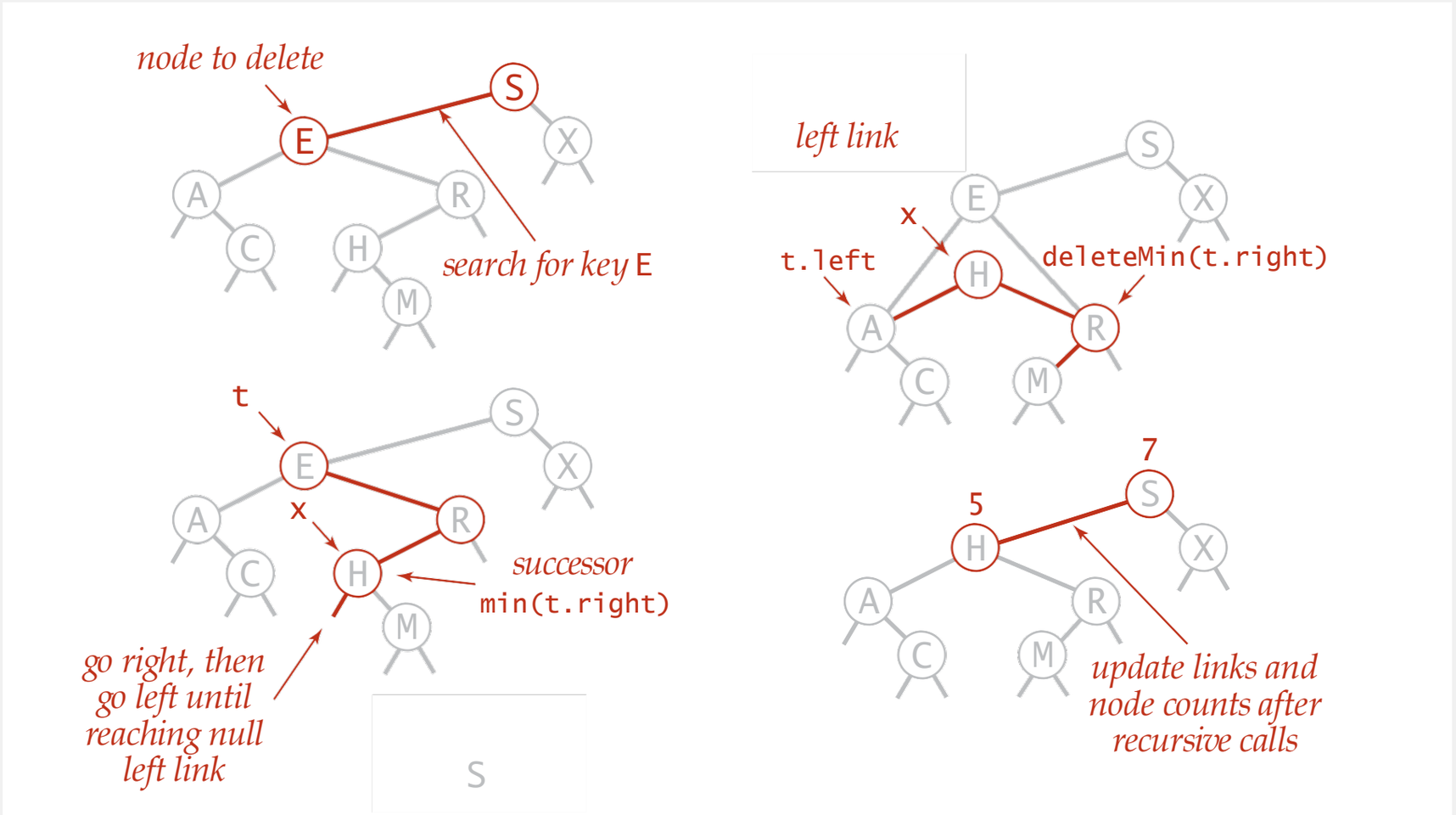


Hibbard delete

Për të delete një nyje me çelës `k`: search për nyjën `t` që përmbanë çelësin `k`.

Rasti 2. [2 fëmijë]

- Gjindet pasrdhësi `x` i `t`-së. ← x has no left child
- Delete minimumi në subtree të djathtë të t-*së* ← but don't garbage collect x
- Put `x` në vendin e `t`-*së*. ← still a BST



Hibbard delete: Java implementimi

```
public void delete(Key key)
{  root = delete(root, key);  }

private Node delete(Node x, Key key) {
   if (x == null) return null;
   int cmp = key.compareTo(x.key);
   if      (cmp < 0) x.left  = delete(x.left,  key);
   else if (cmp > 0) x.right = delete(x.right, key);
   else {
      if (x.right == null) return x.left;
      if (x.left  == null) return x.right;

      Node t = x;
      x = min(t.right);
      x.right = deleteMin(t.right);
      x.left = t.left;
   }
   x.count = size(x.left) + size(x.right) + 1;
   return x;
}
```

search for key

no right child

no left child

replace with successor

update subtree counts

ST implementimi: përmbledhje

implementation	guarantee			average case			ordered ops?	operations on keys
	search	insert	delete	search hit	insert	delete		
sequential search (linked list)	N	N	N	$\frac{1}{2}\ N$	N	$\frac{1}{2}\ N$		`equals()`
binary search (ordered array)	$\lg N$	N	N	$\lg N$	$\frac{1}{2}\ N$	$\frac{1}{2}\ N$	✓	`compareTo()`
`BST`	N	N	N	$1.39\ \lg N$	$1.39\ \lg N$	$\sqrt{N}$	✓	`compareTo()`

other operations also become √N
if deletions allowed